

Homework 4

Alex Ojemann

2025-02-27

Problem 1

The full hierarchical model is:

$$P(\beta_{0a}, \sigma_a^2, z_{ia}, Y_{ia}^* | Y_{ia} \geq 10, n_{ia}) \propto$$

$$\prod_{i=1}^{Ns} P(Y_{ia} | \lambda_{ia}) P(Y_{ia}^* | \lambda_{ia}) P(\lambda_{ia} | \beta_{0a}, z_{ia}) P(z_{ia} | \sigma_a^2) P(\beta_{0a}) P(\sigma_a^2).$$

where:

$$P(Y_{ia} | \lambda_{ia}) = \frac{(n_{ia} \lambda_{ia})^{Y_{ia}} e^{-n_{ia} \lambda_{ia}}}{Y_{ia}!}, \quad \text{for } Y_{ia} \geq 10.$$

$$P(Y_{ia}^* | \lambda_{ia}) = \frac{P(Y_{ia}^* | \lambda_{ia})}{P(Y_{ia} < 10 | \lambda_{ia})}, \quad \text{for suppressed values } Y_{ia}^*.$$

$$\log \lambda_{ia} = \beta_{0a} + z_{ia}, \quad \Rightarrow \quad P(\lambda_{ia} | \beta_{0a}, z_{ia}) = \delta(\lambda_{ia} - e^{\beta_{0a} + z_{ia}}).$$

$$P(z_{ia} | \sigma_a^2) = \frac{1}{\sqrt{2\pi\sigma_a^2}} e^{-z_{ia}^2 / (2\sigma_a^2)}.$$

$$P(\beta_{0a}) = \frac{1}{\sqrt{2\pi \cdot 10,000}} e^{-\beta_{0a}^2 / (2 \cdot 10,000)}.$$

$$P(\sigma_a^2) = \frac{0.001^{0.001}}{\Gamma(0.001)} (\sigma_a^2)^{-1.001} e^{-0.001/\sigma_a^2}.$$

Problem 2

Full Conditional for β_{0a} :

$$P(\beta_{0a} | Y, z, \sigma_a^2) \propto P(Y | \beta_{0a}, z) P(\beta_{0a}).$$

$$P(Y_{ia} | \lambda_{ia}) = \frac{(n_{ia} e^{\beta_{0a} + z_{ia}})^{Y_{ia}} e^{-n_{ia} e^{\beta_{0a} + z_{ia}}}}{Y_{ia}!}.$$

$$\sum_{i=1}^{Ns} [Y_{ia}(\beta_{0a} + z_{ia}) - n_{ia}e^{\beta_{0a} + z_{ia}}] .$$

$$P(\beta_{0a}) = \frac{1}{\sqrt{2\pi\tau_a^2}} e^{-\frac{\beta_{0a}^2}{2\tau_a^2}} .$$

$$P(\beta_{0a}|Y, z, \sigma_a^2) \propto e^{\sum_{i=1}^{Ns} Y_{ia}(\beta_{0a} + z_{ia}) - n_{ia}e^{\beta_{0a} + z_{ia}}} \times e^{-\frac{\beta_{0a}^2}{2\tau_a^2}} .$$

This requires the use of Metropolis steps because it's not of the same form as its prior.

Full Conditional for z_{ia} :

$$P(z_{ia}|Y, \beta_{0a}, \sigma_a^2) \propto P(Y_{ia}|\lambda_{ia})P(z_{ia}|\sigma_a^2) .$$

$$P(Y_{ia}|\lambda_{ia}) \propto e^{Y_{ia}(\beta_{0a} + z_{ia}) - n_{ia}e^{\beta_{0a} + z_{ia}}} .$$

$$P(z_{ia}|\sigma_a^2) = \frac{1}{\sqrt{2\pi\sigma_a^2}} e^{-z_{ia}^2/(2\sigma_a^2)} .$$

$$P(z_{ia}|Y, \beta_{0a}, \sigma_a^2) \propto e^{Y_{ia}(\beta_{0a} + z_{ia}) - n_{ia}e^{\beta_{0a} + z_{ia}}} e^{-\frac{z_{ia}^2}{2\sigma_a^2}} .$$

This requires the use of Metropolis steps because it's not of the same form as its prior.

Full Conditional for σ_a^2 :

$$P(\sigma_a^2|z) \propto P(z|\sigma_a^2)P(\sigma_a^2) .$$

$$P(z|\sigma_a^2) = \prod_{i=1}^{Ns} \frac{1}{\sqrt{2\pi\sigma_a^2}} e^{-z_{ia}^2/(2\sigma_a^2)} ,$$

$$P(\sigma_a^2) = \frac{b^a}{\Gamma(a)} (\sigma_a^2)^{-a-1} e^{-b/\sigma_a^2}, \quad \text{where } a = 0.001, b = 0.001 .$$

$$\sigma_a^2|z \sim \text{Inverse-Gamma} \left(0.001 + \frac{Ns}{2}, 0.001 + \frac{1}{2} \sum_{i=1}^{Ns} z_{ia}^2 \right) .$$

This can be sampled from directly because it's of the same form as its prior.

Problem 3

Map:

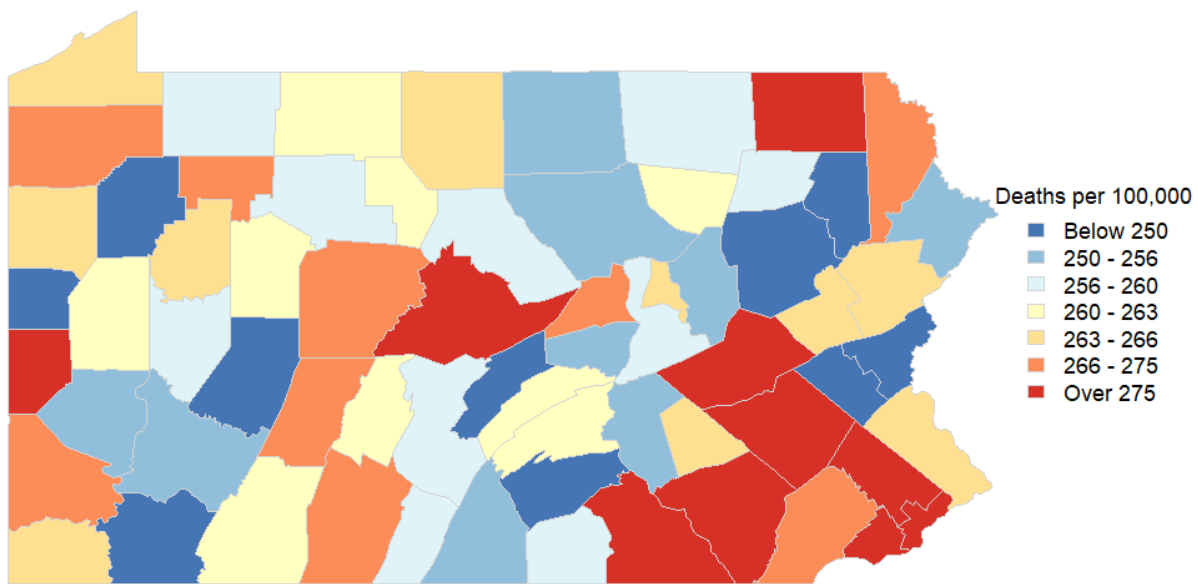


Figure 1: Stroke Mortality Estimates by County in PA

```
#https://wonder.cdc.gov/controller/saved/D140/D34F844
rm(list=ls())
#First we read in the data and define a few things...
stroke=read.table('2016_PA_stroke_total.txt',sep='\t',
                  stringsAsFactors=FALSE,header=TRUE)
Ng=3           #three age groups
alabs=unique(stroke$Age.Group.Code)
Ns=67         #67 counties
clabs=unique(stroke$County)

#Next we organize things a bit...
Y=array(stroke$Deaths,dim=c(Ng,Ns))
n=array(stroke$Population,dim=c(Ng,Ns))

#####
#####
#per part 4, all Y's below 10 have been suppressed
#####
thres=10      #Suppression threshold
dY=!is.na(Y)  #0 for suppressed, 1 for observed
nsupp=apply(!dY,1,sum) #how many suppressed per age
#note: we do not know the true Y's
#       CDC did the suppression, not me

#####
#####
#insert your prior info here
#####
lam0=c(75,250,1000)/100000
```

```

total_pop = sum(n)
age_proportions = apply(n, 1, sum) / total_pop
n0= age_proportions * 10000
Y0= n0 * lam0
tau_a2 = 10000 # Variance for beta_0a prior
a0 = 0.001 # For inverse-gamma prior on sigma_a^2
b0 = 0.001 # For inverse-gamma prior on sigma_a^2

#####
#####
#initialize your Gibbs sampler here
nsims=10000
lami=array(dim=c(Ng,Ns,nsims))
for(a in 1:Ng){
  lami[a,,1]= lam0[a]
  Y[a,!dY[a,]]= Y0[a]
  #Note: the preceding line assumes we don't care
  #      what the posterior dist of the missing
  #      Y's looks like -- I'm just plugging the
  #      current guesses directly into my data vector
}
# Initialize Gibbs sampler storage
beta_0a = matrix(0, nrow = Ng, ncol = nsims) # Store beta_0a samples
sigma_a2 = matrix(1, nrow = Ng, ncol = nsims) # Store sigma_a^2 samples
z = array(0, dim = c(Ng, Ns, nsims)) # Store latent z_ia samples

for(it in 2:nsims){
  for(a in 1:Ng){

#####
# ADDRESS SUPPRESSED Y HERE
#####

    for(i in 1:Ns){
      if(!dY[a, i]){
        lambda_curr = lami[a, i, it-1]

        # Truncated Poisson sampling via inverse transform
        p_cdf = ppois(9, lambda_curr * n[a, i]) # CDF for Y < 10
        u = runif(1) * p_cdf
        Y[a, i] = qpois(u, lambda_curr * n[a, i])
      }
    }

#####
# SAMPLE Z_{ia} USING METROPOLIS
#####

    for(i in 1:Ns) {
      z_proposed = rnorm(1, mean = z[a, i, it - 1], sd = 0.1) # Proposal step

      # Compute log acceptance ratio
      log_r = Y[a, i] * z_proposed - n[a, i] * exp(beta_0a[a, it - 1] + z_proposed) -

```

```

      (Y[a, i] * z[a, i, it - 1] - n[a, i] * exp(beta_0a[a, it - 1] + z[a, i, it - 1])) -
      (z_proposed^2 / (2 * sigma_a2[a, it - 1])) + (z[a, i, it - 1]^2 / (2 * sigma_a2[a, it - 1]))

    # Accept or reject proposal
    if (log(runif(1)) < log_r) {
      z[a, i, it] = z_proposed
    } else {
      z[a, i, it] = z[a, i, it - 1]
    }
  }
}

#####
# SAMPLE BETA_{0a} USING METROPOLIS
#####

beta_proposed = rnorm(1, mean = beta_0a[a, it - 1], sd = 0.1)
log_r_beta = sum(Y[a, ] * beta_proposed - n[a, ] * exp(beta_proposed + z[a, , it])) -
  sum(Y[a, ] * beta_0a[a, it - 1] - n[a, ] * exp(beta_0a[a, it - 1] + z[a, , it])) -
  (beta_proposed^2 / (2 * tau_a2)) + (beta_0a[a, it - 1]^2 / (2 * tau_a2))

if (log(runif(1)) < log_r_beta) {
  beta_0a[a, it] = beta_proposed
} else {
  beta_0a[a, it] = beta_0a[a, it - 1]
}

#####
# SAMPLE SIGMA_A^2 FROM INVERSE-GAMMA
#####

shape_new = a0 + Ns / 2
scale_new = b0 + sum(z[a, , it]^2) / 2
sigma_a2[a, it] = 1 / rgamma(1, shape = shape_new, rate = scale_new)

#####
# ESTIMATE LAMBDA_{ia}
#####

for(i in 1:Ns){
  lami[a, i, it] = exp(beta_0a[a, it] + z[a, i, it])
}
}
}

#####
# Compute posterior summaries
#####
# Number of burn-in samples
burnin = 2500

#####

```

```

# Get posterior samples of age-adjusted rates
#####
aalami = array(dim = c(Ns, nsims))
std_pop = apply(n, 1, sum)
w_a = std_pop / sum(std_pop)
for(i in 1:Ns){
  aalami[i,]= colSums(w_a * lami[, i, ])
}

#####
# Calculate posterior medians
#####
beta_0a_med = apply(beta_0a[, (burnin+1):nsims], 1, median)
z_med = apply(z[, , (burnin+1):nsims], c(1,2), median)
aa.med = apply(aalami[, (burnin+1):nsims], 1, median)

print("Median Beta_0a")

## [1] "Median Beta_0a"

print(beta_0a_med)

## [1] -7.232831 -5.905703 -4.650413

print("Median z_ia")

## [1] "Median z_ia"

print(z_med)

##           [,1]      [,2]      [,3]      [,4]      [,5]      [,6]
## [1,] -0.034706603 -0.04264425 -0.008115352 0.04029658 0.006436821 0.07726054
## [2,] -0.006100096 -0.01879684 0.007483299 0.02161674 -0.021976727 0.12756092
## [3,] 0.010994158 -0.05979071 -0.028709047 0.04663670 0.077178985 0.25199879
##           [,7]      [,8]      [,9]      [,10]     [,11]
## [1,] 0.07352541 -0.003154791 -0.0727246885 -0.02450910 0.02715184
## [2,] -0.02787597 -0.020735473 -0.0005521397 -0.02672190 0.01407595
## [3,] -0.03546360 -0.008487612 0.0285083977 0.01765488 -0.00884656
##           [,12]     [,13]     [,14]     [,15]     [,16]
## [1,] 0.001469297 -0.01474145 -0.021918506 -0.06456456 0.001058457
## [2,] -0.001318197 -0.02765947 -0.007420708 -0.04192437 -0.004528076
## [3,] -0.001978224 0.04850267 0.139459937 0.09108391 0.035048687
##           [,17]     [,18]     [,19]     [,20]     [,21]     [,22]
## [1,] -0.009344648 0.009020997 -0.007194746 -0.02326813 -0.05873453 0.05453541
## [2,] 0.006539274 -0.005804574 -0.022098209 0.03173158 -0.01957203 -0.02227609
## [3,] 0.042180288 -0.039089369 -0.022851415 0.09187213 -0.10289725 -0.11916119
##           [,23]     [,24]     [,25]     [,26]     [,27]
## [1,] 0.20116924 -0.0110348015 -0.03707208 -0.06914912 0.0009640397
## [2,] 0.05582726 -0.0003992441 0.02324383 -0.02500148 -0.0032687651
## [3,] 0.16527616 -0.0310581236 0.01978199 -0.07227108 0.0012461017
##           [,28]     [,29]     [,30]     [,31]     [,32]
## [1,] 0.001825331 -0.011820673 -0.011100986 0.002516841 -0.009540406
## [2,] -0.044344733 -0.001175328 -0.002937966 -0.005840442 -0.006808261
## [3,] -0.057108565 -0.002456924 0.038508131 0.010707859 -0.060968536
##           [,33]     [,34]     [,35]     [,36]     [,37]
## [1,] -0.003867704 -0.0003999762 -0.05403645 0.05226616 -0.031720232

```

```
## [2,] 0.038927648 0.0002019881 0.01425673 0.02776873 -0.002320927
## [3,] -0.014438126 -0.0211629781 -0.09953761 0.06397442 -0.092882258
##      [,38]      [,39]      [,40]      [,41]      [,42]      [,43]
## [1,] 0.029850710 -0.10611996 -0.02324398 -0.03696756 -0.007109965 -0.036061929
## [2,] -0.004634924 -0.07559928 -0.01707564 -0.02172367 -0.005936418 0.009762428
## [3,] -0.009049265 -0.09556150 -0.07899930 -0.04291123 -0.061196066 0.030954626
##      [,44]      [,45]      [,46]      [,47]      [,48]      [,49]
## [1,] -0.004473286 -0.01608197 -0.06842254 -0.001711747 -0.05076605 -0.02404596
## [2,] -0.018634048 0.01566328 0.13568943 -0.001665215 -0.02748740 -0.01509313
## [3,] -0.082791331 0.01430438 0.17780613 -0.005054785 -0.22389747 0.03853200
##      [,50]      [,51]      [,52]      [,53]      [,54]
## [1,] -0.002328328 0.259673904 -0.001751067 0.004298119 0.077714744
## [2,] -0.003241251 0.107502175 -0.011608793 -0.003329942 0.056347402
## [3,] -0.005878022 0.001789481 -0.015416686 -0.002015928 -0.007420918
##      [,55]      [,56]      [,57]      [,58]      [,59]
## [1,] -0.002952213 0.05609491 0.004289832 -0.0007952657 -0.005189321
## [2,] -0.003833069 0.02505178 -0.002063269 -0.0095817756 -0.011269136
## [3,] -0.024427811 -0.09446310 -0.001096007 0.0918798803 -0.045837476
##      [,60]      [,61]      [,62]      [,63]      [,64]      [,65]
## [1,] 4.114933e-05 -0.00308553 0.01116944 -0.15074662 -0.002173651 0.04869861
## [2,] -1.080539e-03 -0.01267070 -0.01285768 -0.02268712 -0.016329126 -0.03382058
## [3,] 4.564442e-02 -0.07105370 -0.06449831 0.12921687 0.048110490 -0.07243046
##      [,66]      [,67]
## [1,] 0.003165500 0.11845362
## [2,] 0.001323637 -0.02486258
## [3,] -0.010467118 0.14713825
```

```
print("Median Lambda_ia")
```

```
## [1] "Median Lambda_ia"
```

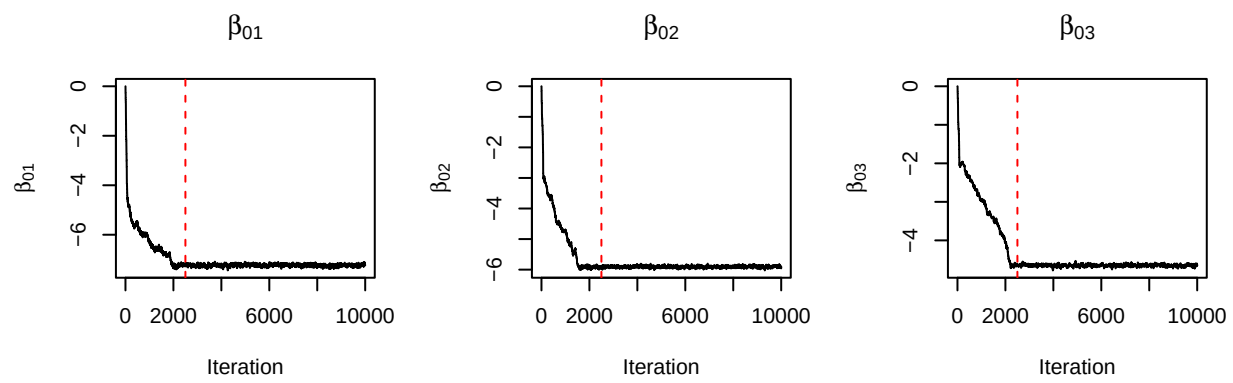
```
print(aa.med)
```

```
## [1] 0.002645970 0.002523578 0.002611563 0.002752318 0.002745493 0.003209250
## [7] 0.002608096 0.002608691 0.002653999 0.002631129 0.002660532 0.002647758
## [13] 0.002691424 0.002848654 0.002719582 0.002697499 0.002717791 0.002593969
## [19] 0.002585538 0.002813327 0.002461728 0.002488823 0.003042424 0.002601583
## [25] 0.002678510 0.002494955 0.002649589 0.002527077 0.002637097 0.002698728
## [31] 0.002664224 0.002552681 0.002666912 0.002616492 0.002498775 0.002782802
## [37] 0.002498796 0.002644849 0.002404289 0.002510144 0.002545223 0.002557382
## [43] 0.002687515 0.002517380 0.002675527 0.003012201 0.002630840 0.002306799
## [49] 0.002679060 0.002636577 0.002852625 0.002614194 0.002642626 0.002721356
## [55] 0.002610669 0.002568595 0.002652218 0.002773292 0.002571070 0.002711601
## [61] 0.002534026 0.002548691 0.002767210 0.002707494 0.002536273 0.002634636
## [67] 0.002908539
```

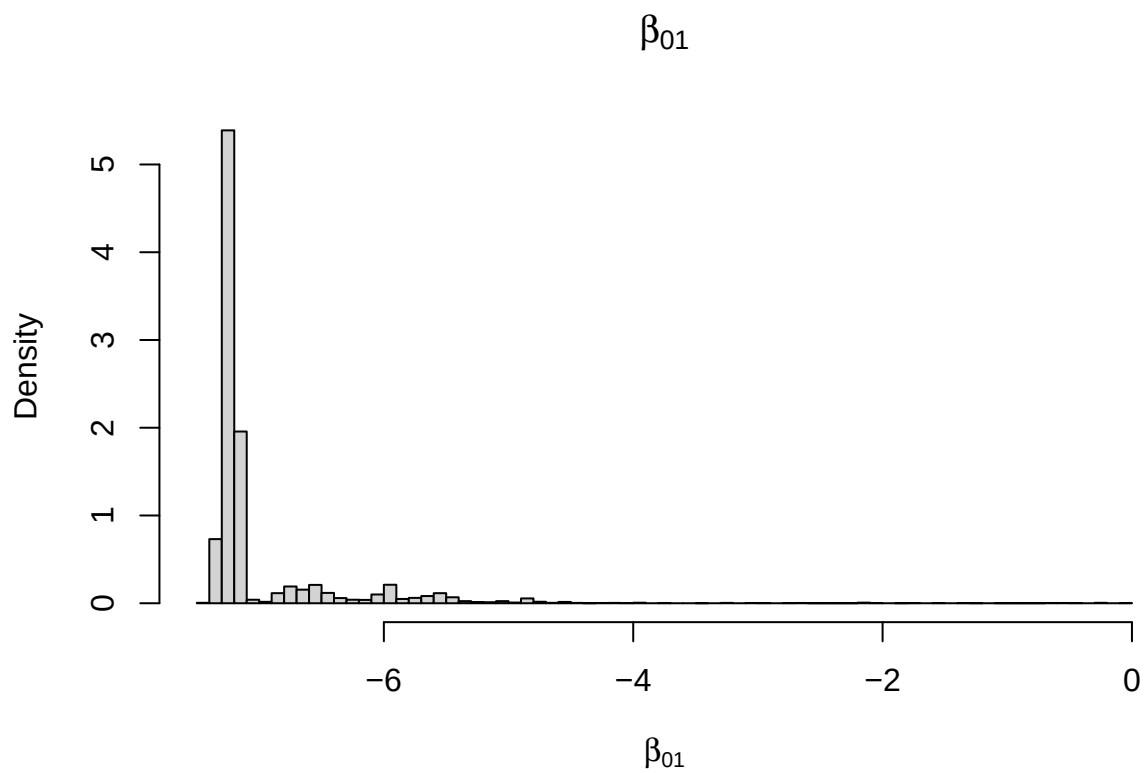
```
# Plots
```

```
par(mfrow = c(2, 3))
```

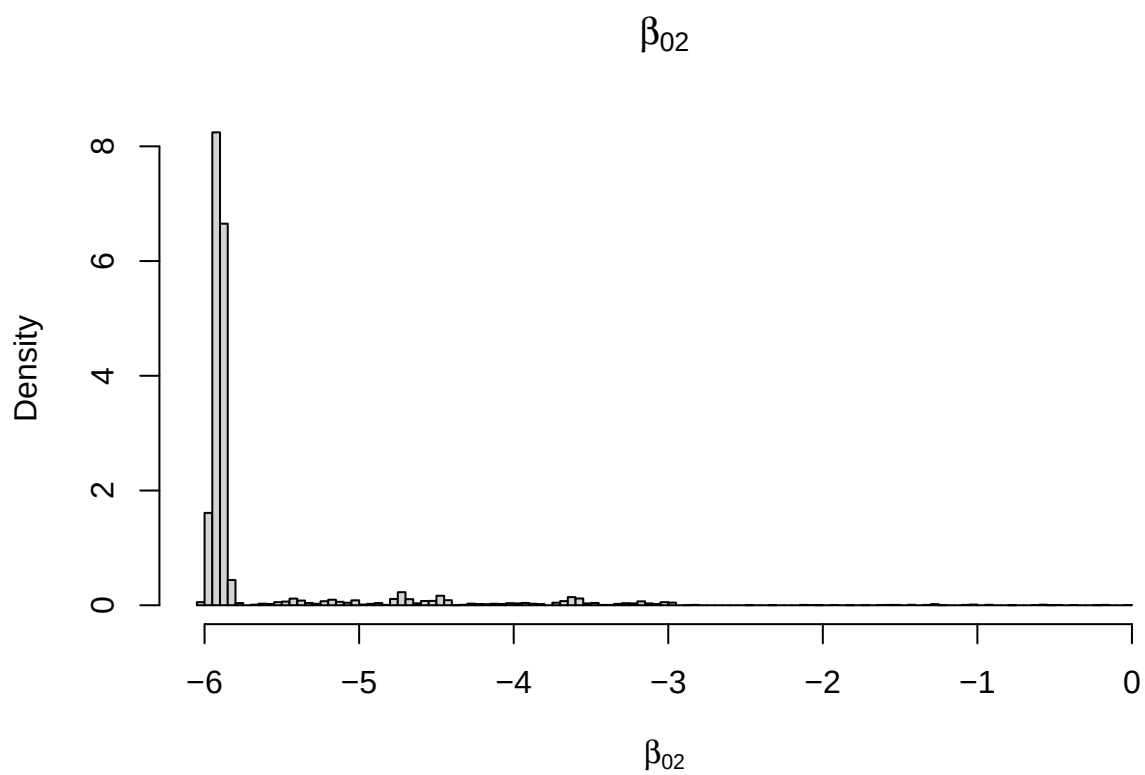
```
plot(beta_0a[1, ], type = 'l', main = expression(beta[0][1]), xlab = "Iteration", ylab = expression(beta[0][1]))
abline(v = burnin, col = "red", lty = 2) # Mark burn-in threshold
plot(beta_0a[2, ], type = 'l', main = expression(beta[0][2]), xlab = "Iteration", ylab = expression(beta[0][2]))
abline(v = burnin, col = "red", lty = 2)
plot(beta_0a[3, ], type = 'l', main = expression(beta[0][3]), xlab = "Iteration", ylab = expression(beta[0][3]))
abline(v = burnin, col = "red", lty = 2)
```



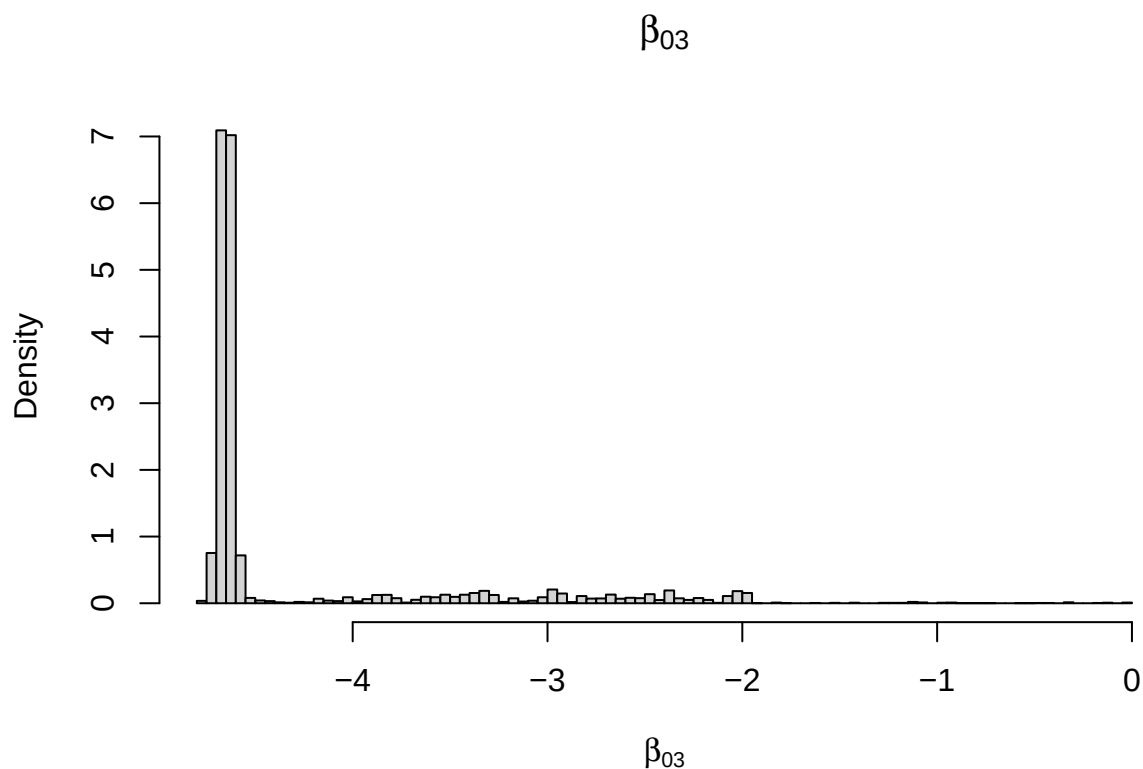
```
hist(beta_0a[1, ], breaks = 100, freq = FALSE, main = expression(beta[0][1]), xlab = expression(beta[0]
```



```
hist(beta_0a[2, ], breaks = 100, freq = FALSE, main = expression(beta[0][2]), xlab = expression(beta[0]
```



```
hist(beta_0a[3, ], breaks = 100, freq = FALSE, main = expression(beta[0][3]), xlab = expression(beta[0]
```

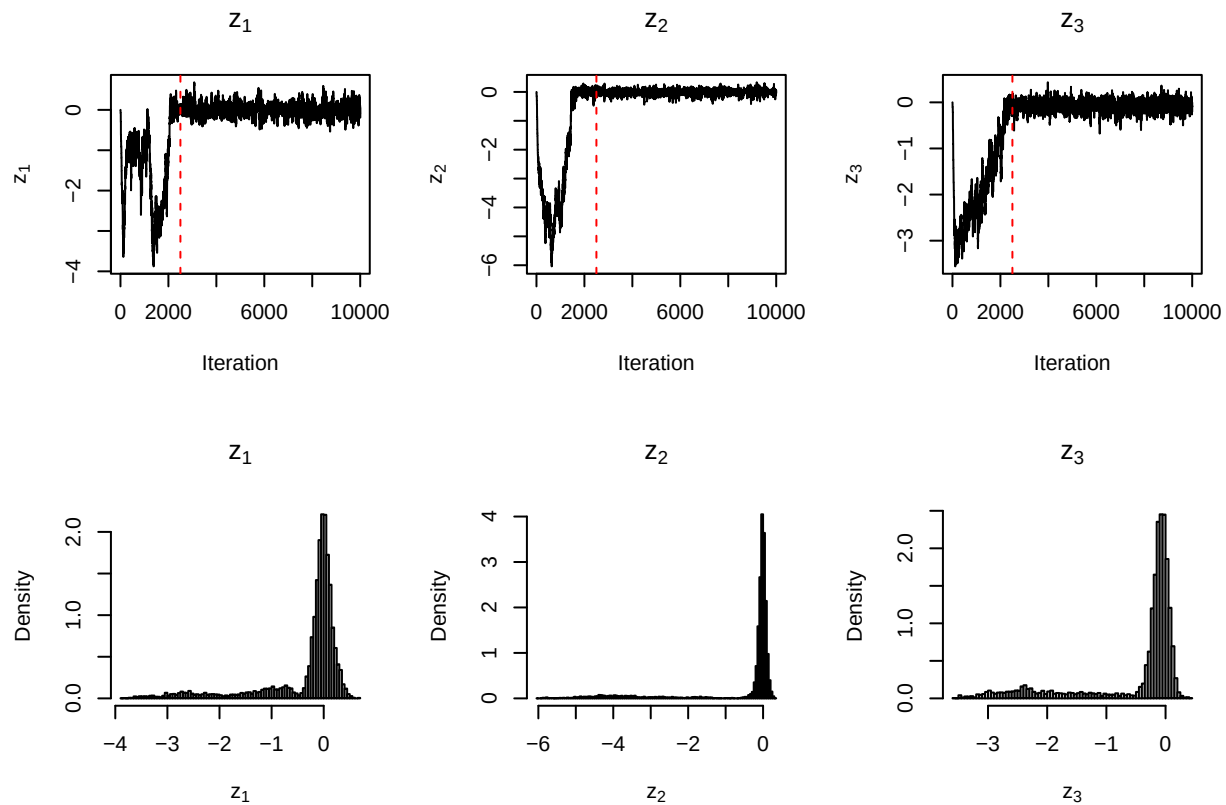


```
par(mfrow = c(2, 3))

# Select a random county for z
random_county = sample(1:Ns, 1)

plot(z[1, random_county, ], type = 'l', main = expression(z[1]), xlab = "Iteration", ylab = expression(
abline(v = burnin, col = "red", lty = 2)
plot(z[2, random_county, ], type = 'l', main = expression(z[2]), xlab = "Iteration", ylab = expression(
abline(v = burnin, col = "red", lty = 2)
plot(z[3, random_county, ], type = 'l', main = expression(z[3]), xlab = "Iteration", ylab = expression(
abline(v = burnin, col = "red", lty = 2)

hist(z[1, random_county, ], breaks = 100, freq = FALSE, main = expression(z[1]), xlab = expression(z[1]
hist(z[2, random_county, ], breaks = 100, freq = FALSE, main = expression(z[2]), xlab = expression(z[2]
hist(z[3, random_county, ], breaks = 100, freq = FALSE, main = expression(z[3]), xlab = expression(z[3]
```



```
#####
#####
#THE BELOW CODE SHOULD BE LEFT AS-IS!
#IT ASSUMES YOU NAMED
#THE POSTERIOR MEDIANS OF THE AGE-ADJUSTED RATES
#"aamed" USING THE CODE ABOVE,
#AND WILL CREATE A MAP "PAmmap.png"
#THAT WILL BE SAVED TO YOUR CURRENT DIRECTORY
#####

install.packages(c("sp", "rgdal", "rgeos", "maps", "RColorBrewer")) # I added this to install packages

load('penn.rdata')
install.packages(c('maps', 'RColorBrewer'))
library(maps)
library(RColorBrewer)
ncols=7
cols=brewer.pal(ncols, 'RdYlBu')[ncols:1]
tcuts=quantile(aa.med*100000, 1:(ncols-1)/ncols)
tcolb=array(rep(aa.med*100000, each=ncols-1) > tcuts,
             dim=c(ncols-1, Ns))
tcol =apply(tcolb, 2, sum)+1

png('PAmmap.png', height=520, width=1000)
par(mar=c(0,0,0,10), cex=1)
plot(penn, col=cols[tcol], border='lightgray', lwd=.5)
```

```

legend('right',inset=c(-.15,0),xpd=TRUE,
      legend=c(paste(
        c('Below',round(tcuts[-(ncols-1)],0),'Over'),
        c(' ',rep(' - ',ncols-2),' '),
        c(round(tcuts,0),round(tcuts[ncols-1],0)),sep='')),
      fill=cols,title='Deaths per 100,000',bty='n',cex=1.5,
      border='lightgray')
dev.off()

```